

Memória, E/S e GPU na Prática

Integrante	RA
Leonardo Lavras	23008784
Lucas Henrique Koda	20018156
João Vitor de Moraes	20068995
Victor Marcelo Camargo	21007132

Campinas, junho de 2025

Identificação da Máquina

Componente	Especificação
Dispositivo	DESKTOP-D29RD9P
Processador	AMD Ryzen 5 5500 — 6 núcleos / 12 threads, 3,60 GHz (Zen 3)
Cache L1d	32 KiB por núcleo (192 KiB total)
Cache L2	512 KiB por núcleo (3 MiB total)
Cache L3	16 MiB compartilhada
Memória RAM	16,0 GB (15,9 GB utilizável)
Armazenamento	SSD NVMe ~447 GB
Placa gráfica	NVIDIA GeForce RTX 4060 — 8 GB GDDR6
Sistema	Windows 11 x64 / Ubuntu 24.04 via WSL2

Ferramentas: gcc 13, fio, Python 3.12 + matplotlib (WSL2); WebGPU via Chrome (Atividade 3).
Auxílio de IA (Claude — Anthropic) utilizado para depuração e formatação do relatório.

Atividade 1 — Hierarquia de Memória (Cache)

Metodologia

O benchmark *cachebench.c* percorre arrays de tamanhos crescentes — de 1 KB a 256 MB, dobrando a cada iteração — e mede o tempo médio de acesso em nanossegundos. O acesso é feito com *stride* fixo de 64 bytes, equivalente ao tamanho de uma linha de cache, forçando o processador a buscar dados em posições distintas sem reutilizar o mesmo bloco. Para cada tamanho são realizadas 5 repetições, registrando o melhor tempo. O código foi compilado com `gcc -O2` e usa `volatile` para impedir que o compilador elimine os acessos.

Comandos executados

```
gcc -O2 -o cachebench src/cache/cachebench.c
./cachebench > results/resultado_cache.csv
python3 scripts/plot_cache.py results/resultado_cache.csv
results/grafico_cache.png
```

Resultados

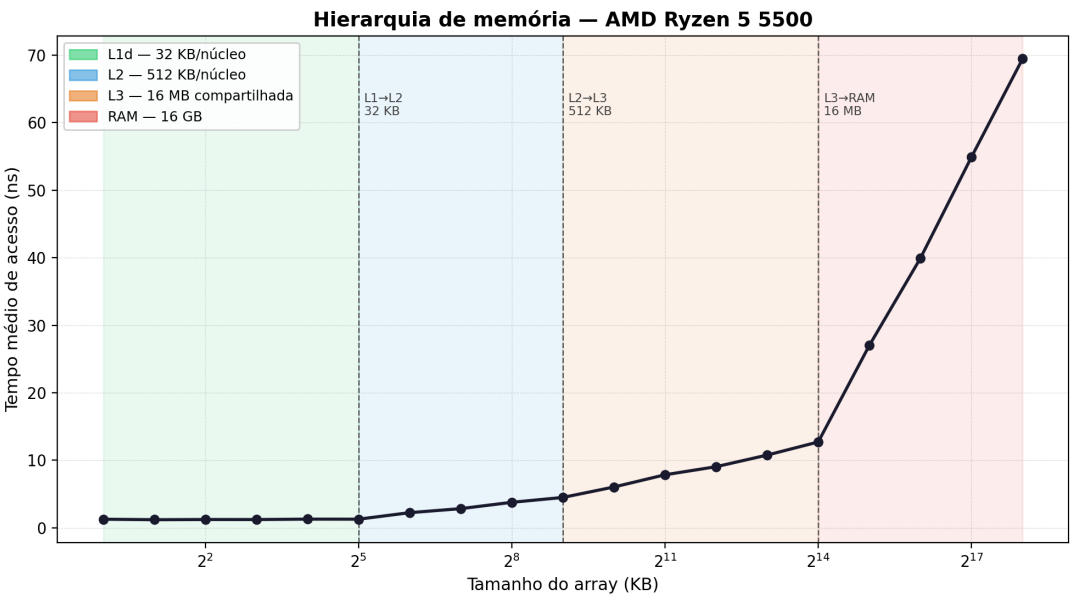


Figura 1 — Tempo de acesso à memória em função do tamanho do array. AMD Ryzen 5 5500, stride de 64 bytes, 20 milhões de acessos por medição.

Tamanho do array	Latência medida (ns)	Nível de cache
1 KB	~1,25	L1d
32 KB	~1,26	L1d (limite — 32 KB/núcleo)
64 KB	~2,19	L2
512 KB	~4,41	L2 (limite — 512 KB/núcleo)

1 MB	~5,87	L3
16 MB	~12,22	L3 (limite — 16 MB total)
32 MB	~26,63	RAM
256 MB	~69,43	RAM

Tabela 1 — Latência de acesso por nível da hierarquia de memória.

Análise

O gráfico revela quatro patamares distintos, cada um correspondendo a um nível da hierarquia de memória do Ryzen 5 5500. Enquanto o array cabe no cache L1d (até 32 KB por núcleo), a latência permanece em torno de 1,2–1,3 ns, praticamente a velocidade do próprio processador. A partir de 64 KB, os dados passam a residir no L2 (512 KB por núcleo) e o tempo sobe para a faixa de 2 a 4,5 ns.

O terceiro degrau ocorre ao ultrapassar 512 KB, quando o array transborda para o cache L3 (16 MB compartilhados): a latência cresce progressivamente até cerca de 12 ns. Por fim, arrays maiores que 16 MB forçam acessos à RAM, elevando a latência para 27–70 ns — até 55 vezes mais lento que o L1. Identificamos três níveis de cache (L1, L2, L3) mais a RAM, totalizando quatro patamares. Os degraus aparecem porque cada nível tem capacidade finita: quando o array não cabe mais no nível atual, o processador passa a buscar dados no nível seguinte, que é mais lento.

Atividade 2 — E/S e Armazenamento

Metodologia

Utilizou-se o **fio** (Flexible I/O Tester) para medir o desempenho do SSD NVMe em dois cenários. A flag `--direct=1` foi utilizada em ambos para contornar o cache de página do sistema operacional, garantindo que as medições reflitam o desempenho real do dispositivo. O primeiro teste realiza leitura sequencial com blocos de 1 MB, simulando streaming de vídeo ou cópia de arquivos grandes. O segundo realiza leitura aleatória com blocos de 4 KB e profundidade de fila de 32 (`iodepth=32`), simulando o padrão de acesso de um banco de dados relacional.

Comandos executados

```
fallocate -l 1G results/testfile
fio --name=seqread --rw=read --bs=1M --direct=1 --runtime=20 --time_based
fio --name=randread --rw=randread --bs=4k --direct=1 --runtime=20 --time_based
--iodepth=32
```

Resultados

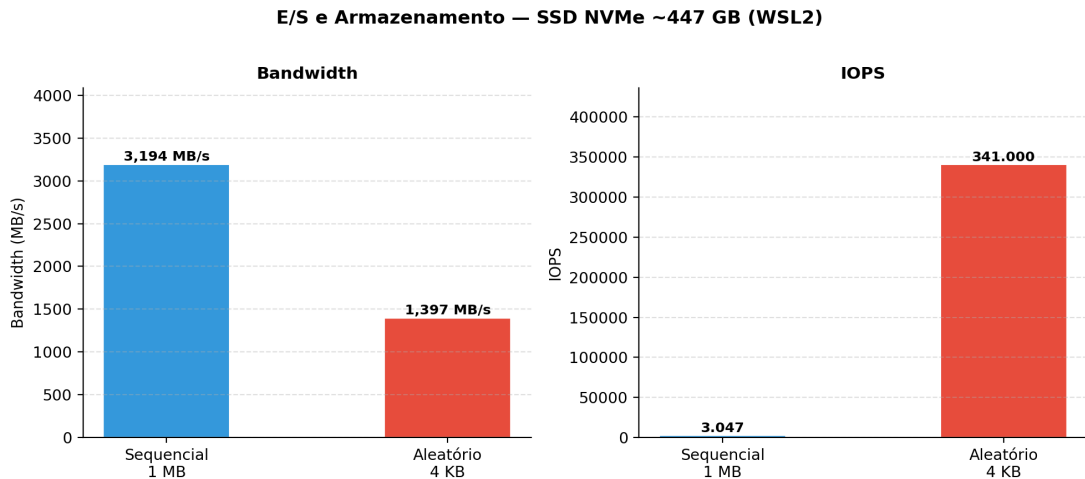


Figura 2 — Bandwidth (MB/s) e IOPS nos dois cenários de E/S. SSD NVMe ~447 GB, --direct=1, runtime=20 s.

Cenário	Block size	Bandwidth	IOPS	iodepth
Leitura sequencial	1 MB	3.194 MB/s	3.047	1
Leitura aleatória	4 KB	1.397 MB/s	341.000	32

Tabela 2 — Resultados do fio nos dois cenários de leitura.

Análise

A leitura sequencial com blocos de 1 MB entregou 3.194 MB/s — próximo ao limite teórico de SSDs NVMe de entrada. Isso ocorre porque o controlador consegue pré-buscar (*read-ahead*) os próximos blocos enquanto entrega o atual, aproveitando ao máximo a largura de banda do barramento PCIe. Já na leitura aleatória com blocos de 4 KB, mesmo com 341.000 IOPS, cada operação move apenas 4 KB, resultando em 1.397 MB/s de bandwidth efetivo. Em HDDs tradicionais essa diferença seria catastrófica: o cabeçote mecânico precisaria se reposicionar fisicamente a cada acesso, podendo cair abaixo de 1 MB/s.

Essa distinção tem implicações diretas de projeto: um servidor de streaming lê arquivos de forma contínua e se beneficia do alto bandwidth sequencial, enquanto um banco de dados relacional acessa páginas em posições arbitrárias e depende de alto IOPS para atender múltiplas consultas simultâneas.

Atividade 3 — GPU pelo Navegador (WebGPU)

Metodologia

A comparação foi realizada com o arquivo `src/webgpu/matrix_webgpu.html` no Chrome, utilizando a GPU NVIDIA GeForce RTX 4060 (8 GB) via API WebGPU. A versão CPU implementa multiplicação de matrizes em JavaScript puro com três loops aninhados ($O(n^3)$). A versão GPU utiliza um *compute shader* WGSL com *workgroup* de 8×8 threads, calculando cada elemento da matriz em paralelo. O cronômetro foi parado somente após `device.queue.onSubmittedWorkDone()`, garantindo que a GPU tenha concluído todo o processamento antes da medição.

Resultados

CPU vs GPU — WebGPU | Ryzen 5 5500 + RTX 4060

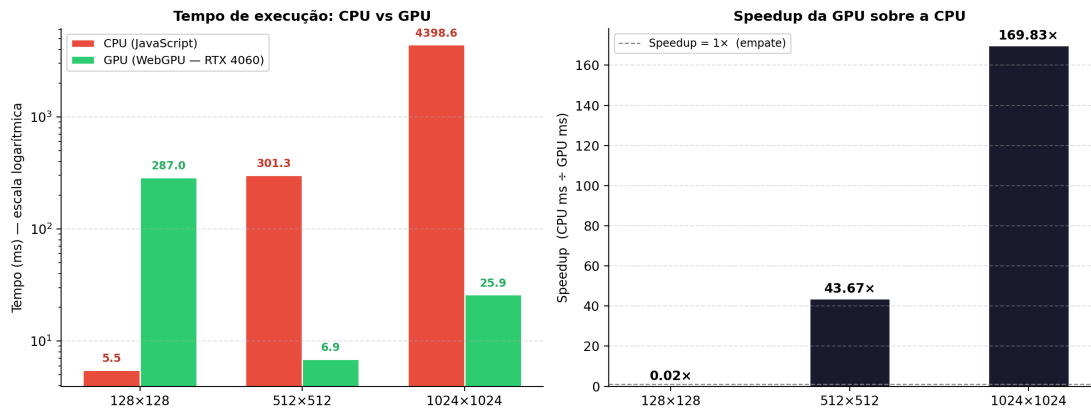


Figura 3 — Tempo de execução (escala logarítmica) e speedup da GPU sobre a CPU. RTX 4060 via WebGPU no Chrome.

Tamanho	CPU (ms)	GPU (ms)	Speedup
128 × 128	5,50	287,00	0,02×
512 × 512	301,30	6,90	43,67×
1024 × 1024	4.398,60	25,90	169,83×

Tabela 3 — Resultados reais da multiplicação de matrizes CPU vs. GPU (WebGPU, Chrome).

Captura de tela — execução real no navegador

Multiplicação de matrizes - CPU x GPU

Teste simples usando JavaScript para CPU e WebGPU para GPU.

Rodar testes

Finalizado.

Tamanho	CPU (ms)	GPU (ms)	Speedup
128x128	5.50	287.00	0.02x
512x512	301.30	6.90	43.67x
1024x1024	4398.60	25.90	169.83x

```

[
  {
    "n": 128,
    "cpuTime": 5.5,
    "gpuTime": 287,
    "speedup": 0.021637638620289
  },
  {
    "n": 512,
    "cpuTime": 301.299999999999836,
    "gpuTime": 6.90000000023283,
    "speedup": 43.66666666517635
  },
  {
    "n": 1024,
    "cpuTime": 4398.5999999999977,
    "gpuTime": 25.90000000023283,
    "speedup": 169.83011582996227
  }
]

```

Figura 4 — Captura de tela da execução real do benchmark WebGPU no Chrome (DESKTOP-D29RD9P, RTX 4060). Os valores coincidem com a Tabela 3.

Análise

Os resultados demonstram de forma clara o ponto de cruzamento entre CPU e GPU. Para a matriz 128x128, a CPU foi mais rápida (5,50 ms contra 287 ms da GPU), com speedup de 0,02x. Esse comportamento era esperado: a GPU precisa de tempo para inicializar buffers, compilar o shader e transferir dados para a VRAM — overhead fixo que não compensa para operações pequenas.

A partir de 512×512 o cenário se inverte: a GPU executou em 6,90 ms enquanto a CPU levou 301,30 ms (speedup de 43,67×). Para 1024×1024 a vantagem foi ainda maior — GPU em 25,90 ms contra 4.398,60 ms da CPU, resultando em speedup de 169,83×. Isso se explica pelo paralelismo massivo da RTX 4060: o shader WGSL despacha milhares de threads simultâneas, cada uma calculando um único elemento da matriz — para 1024×1024, mais de 1 milhão de elementos em paralelo.

O experimento ilustra que a GPU não é universalmente mais rápida — ela vence quando o problema tem paralelismo suficiente para diluir o overhead de configuração. Multiplicação de matrizes grandes é um caso ideal, o que explica por que frameworks como PyTorch e TensorFlow utilizam GPUs como base de seus cálculos.

Referências

1. DREPPER, U. *What Every Programmer Should Know About Memory*. Red Hat, 2007. Disponível em: <<https://lwn.net/Articles/250967/>>. Acesso em: jun. 2025.
2. MIT OpenCourseWare. *6.004 Computation Structures — Aula 14*. Disponível em: <<https://ocw.mit.edu/courses/6-004-computation-structures-spring-2017/pages/c14/>>. Acesso em: jun. 2025.
3. ORACLE LINUX BLOG. *Overview of NVMe Architecture*. Disponível em: <<https://blogs.oracle.com/linux/overview-of-nvme-architecture>>. Acesso em: jun. 2025.
4. fio — Flexible I/O Tester. *fio HOWTO*. Disponível em: <<https://fio.readthedocs.io/>>. Acesso em: jun. 2025.
5. GOOGLE CHROME DEVELOPERS. *Get started with GPU Compute on the web*. Disponível em: <<https://developer.chrome.com/docs/capabilities/web-apis/gpu-compute>>. Acesso em: jun. 2025.
6. WebGPU Fundamentals. Disponível em: <<https://webgpufundamentals.org/>>. Acesso em: jun. 2025.